

Практика 2.2 ДЗ 1. Соревнование по подбору паролей

Технический отчет по выполненной работе

Цель работы	увеличить долю взломанных паролей (cracked rate) на тестовом наборе за счет целевой генерации кандидатов;
Основной подход	эволюция промптов OpenEvolve и управление длиной генерируемых паролей относительно исходного пароля;
Инструменты	PassLLM, OpenEvolve, Qwen1.5B, ChatGPT, PyTorch/CUDA, vLLM.

1. Краткая справка

Во время выполнения первого домашнего задания я встретил много трудностей, нашел одно ключевое решение, позволившее увеличить cracked rate почти в два раза, но больше всего набрал опыта во взаимодействии со сторонними GPU-серверами, анализе сложности программы и особенностях работы с CUDA. Ниже — обо всем по порядку.

2. Ключевые идеи, которые улучшили результат

В начале я экспериментировал с параметрами эволюции системного промпта: менял количество островов, температуру и сам системный промпт OpenEvolve. Однако, глядя на генерируемые пароли, я понял одно: модель Qwen1.5B совершенно не умеет останавливаться или, наоборот, пишет слишком длинный пароль. Решение пришло само: я ограничил длину генерации нового пароля по размеру старого пароля. Это, казалось бы, простое и даже потенциально вредное решение позволило мгновенно повысить cracked rate.

Далее я стал экспериментировать: среди 100 генерируемых новых паролей я настраивал параметры так, что x паролей должны были иметь ту же длину, y — на один токен больше, z — на два токена больше и так далее. Проведя статистический анализ всех пар паролей из train.json с помощью ChatGPT, подобрать эти параметры стало проще. [Ссылка на анализ](#).

3. Стратегия эволюции и подбор ограничений длины

Я запустил тест на нескольких вариантах ограничений. Каждый элемент в списке — это количество генераций определенной длины пароля в порядке [N, N-1, N+1, N-2, N+2, N+3, N+4], где N — длина старого пароля. Проверенные конфигурации приведены ниже.

№	Конфигурация	Комментарий
1	[5, 10, 15, 10, 20, 20, 10]	
2	[5, 0, 20, 0, 28, 26, 21]	
3	[1, 0, 22, 0, 30, 26, 21]	
4	[5, 20, 10, 20, 20, 10, 15]	
5	[3, 0, 21, 0, 29, 26, 21]	выбранный вариант
6	[0, 20, 20, 20, 20, 20, 0]	
7	[20, 0, 20, 20, 20, 20, 0]	
8	[20, 20, 0, 20, 20, 20, 0]	
9	[20, 20, 20, 0, 20, 20, 0]	
10	[20, 20, 20, 20, 0, 20, 0]	
11	[20, 20, 20, 20, 20, 0, 0]	

В качестве дальнейшей генерации был выбран вариант, показавший лучший результат: [3, 0, 21, 0, 29, 26, 21]. В начале этот метод показывал cracked rate 0,4, но спустя 30 циклов эволюции промпта достиг 50% cracked rate, существенно превысив прошлый результат 0,1.

Также слегка увеличить cracked rate помогло использование метапромптинга: системный промпт для OpenEvolve, а также стартовый промпт для Qwen1.5B написал ChatGPT, учитывая специфику работы программы на основе статистического анализа, проведенного выше. Кроме того, переключение `diff_based_evolution` в режим `false` дало небольшой положительный результат: эволюция стала сходиться быстрее.

4. Сложности и технические решения

4.1. Поиск и аренда GPU-сервера

Основной сложностью программы был поиск и аренда GPU-сервера. Необходимые серверы с RTX 4080/4090/5080/5090, а также серверные варианты, часто имели либо плохое соединение, либо стоили не менее 40 рублей в час. Для проверки влияния параметров OpenEvolve на скорость сходимости эволюции и ее предел уходил примерно час, а прогон 40 итераций эволюции для проверки контрольного подхода занимал порядка трех часов. В конце, для генерации `submission.csv`, состоящего из 8 тысяч паролей, ушло 8 часов на RTX 4080.

Для ускорения внедрения своей программы и вычислений я подключил GitHub, настроил `ssh-agent` и VS Code, добавил `hf_token`, подключил `batch_size/num_return_sequences` в параметрах `model.generate`, а затем подключил `vLLM`, реализовал выбор `device` и использовал шаблоны для импорта разных CUDA-совместимых библиотек, например `PyTorch`. В результате скорость базовой настройки сервера и прогона инициализируемых ячеек блокнота сократилась примерно на 20 минут, а итерации эволюции в целом стали проходить быстрее.

4.2. Ограничения длины и EOS_token

Еще одна проблема заключалась в том, что мягкие ограничения сверху заставляли модель генерировать либо пароль с максимальной, либо с нулевой длиной. Проблема крылась в `EOS_token`: модель буквально не умела останавливаться и ставила вместо `EOS_token` точки или другие неподходящие символы. Решение — ввести строгое ограничение снизу и сверху, чтобы модель физически не могла сгенерировать пароль за пределами необходимой длины.

Это решение сразу повысило cracked rate с 0,01 до 0,1. Не стоит путать его с ключевой идеей, описанной выше: в этом случае верхняя и нижняя границы были равны ± 1 от длины старого пароля и фактически все еще не давали модели генерировать пароль такого же размера, как старый пароль.

4.3. Таймауты, память и отдельные процессы

Еще одна проблема, с которой я столкнулся, тоже была связана с `model.generate`, таймаутом у OpenEvolve и `evaluator`. Проблема была в том, что Qwen8B и Qwen1.5B не имели ограничений по времени работы и выходным токенам, а таймаут был слишком мал. Модель эволюции либо не справлялась за 10 минут с генерацией и уходила в таймаут, повторяя этот цикл, либо модель для генерации паролей не умела выгружать себя из памяти стандартными способами. В результате память быстро забивалась, программа уходила в бесконечный троттлинг, а эволюция останавливалась на 4–7-й итерации.

Помогли введение лимитов времени и токенов в параметрах `model.generate`, увеличение таймаутов, а также разделение генерации по отдельным процессам. Также я реализовал принудительное удаление процесса и очистку CUDA-кеша в случае срабатывания таймаута. Кроме этого, каждый новый запуск блокнота подчищает старые процессы, оставшиеся после прошлого запуска. В результате удалось сократить линейный рост потребления памяти, ликвидировать троттлинг и обеспечить полные циклы эволюции вплоть до 60-й итерации и более.

4.4. Запуск на разных ОС и окружениях

Также была проблема в запуске программы на разных ОС. Для тестирования подходов, требующих мощного GPU, я арендовал серверы на Ubuntu, причем из-за специфики работы сервиса они часто работали на разных версиях и сборках. Для тестирования остальных случаев, не требующих больших мощностей, я запускал все локально на MacBook.

В связи с этим возникало множество проблем, связанных с путями к файлам: все было реализовано локально в папке, включая `venv` и отключение символических ссылок при скачивании моделей на Hugging Face. Также возникали проблемы с виртуальным окружением, библиотеками, выбором GPU (MPS/CUDA/CPU), установкой необходимых командных утилит, включая NVIDIA.

Я реализовал кросс-платформенность, автоматическую установку утилит для поддержки NVIDIA GPU, настроил внутреннее окружение так, чтобы оно не конфликтовало с ограничениями Docker-контейнера,

внутри которого запускается моя программа, и логировал ошибки на каждом этапе. Также я запускал серверы только на версиях CUDA выше 13.0 из-за проблемы совместимости библиотек. Это позволило без проблем запускать программу на разных ОС, сборках, драйверах и видеокартах.

5. Итог

Итоговый прирост качества в основном был связан не с усложнением промпта, а с контролем длины генерируемых паролей относительно исходного пароля. Эволюция промптов, метапромптинг и отключение `diff_based_evolution` дополнительно улучшили сходимость, но ключевым оказалось именно ограничение генерации по длине. Отдельный значимый результат работы — воспроизводимая инфраструктура запуска на разных GPU-серверах и локальной машине, с контролем таймаутов, памяти и CUDA-окружения.